AdvNet

Revealing Performance Issues in Network Protocols by Generating Adversarial Environments

Motivation, search design, and practical use cases

Shehab Sarar Ahmed et al. Proc. ACM Netw. 4, CoNEXT2, Article 12 (2026).

Bandwidth and throughput



- Bandwidth is link capacity; throughput is the achieved rate
- Sending too slowly leaves capacity unused
- Sending too quickly builds queues and increases delay

RTT: round-trip time.

What controls TCP sending?

Sending controls

- Congestion window limits unacknowledged data in flight
- Pacing controls packet departure timing

Feedback and recovery

- ACK processing
- Duplicate ACKs and fast retransmit
- Retransmission timeout (RTO)

Why congestion control is hard to test

A changing environment

- Large space of network parameters and temporal patterns
- Protocol state carries across changes in conditions

A changing implementation

- Protocol updates can introduce regressions in unfamiliar environments
- Examples: new BBR versions and multipath support

Existing testing methods

Method	What it provides	Coverage limit
Hand-designed tests	Controlled, interpretable scenarios	What the designer anticipated
Trace replay / testbeds	Evidence from observed conditions	Where and when measurements were collected

Adversarial search adds tests beyond anticipated or sampled conditions.

Simulation and emulation

	Simulation	Emulation
Protocol	Model of behavior	Real implementation
Network	Modeled conditions	Controlled conditions
Main concern	Model fidelity	Execution cost and noise

Repeatability and runtime depend on the model and execution setup.

CC-Fuzz and the move to emulation

CC-Fuzz: GA in ns-3

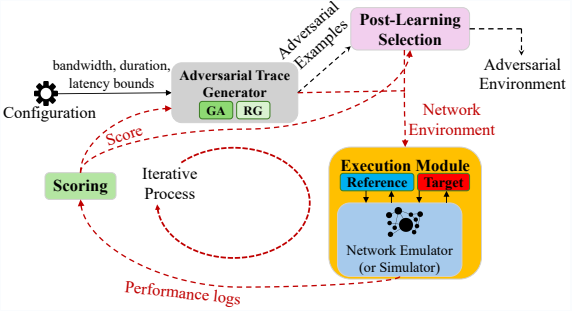
Packet delivery opportunities (PDOs) specify when a link can transmit a packet.

Moving to real-code emulation

- Each candidate costs a real execution
- Search dimension and measurement noise consume the available budget

The AdvNet framework

- Search over candidate network environments
- Compare a target with a meaningful reference
- Use measured performance gaps as feedback



The performance-gap score

$$\text{score} = \frac{R - P}{\max(R, P)}$$

Higher is better: R = reference, P = target
Example: R = 10, P = 2 gives score = 0.8

$$U = t_{\text{coeff}} \frac{\tau}{\tau_{\max}} + (1 - t_{\text{coeff}}) \frac{d_{\min}}{d}$$

τ: throughput; d: average packet delay; τmax and dmin: normalization benchmarks

Throughput weight: 0 = delay only, 0.5 = equal normalized weights, 1 = throughput only

Post-learning selection

Search

Generate and evaluate new candidate traces

PLS

Re-measure promising candidates and select by mean

Independent check

Re-evaluate the returned trace

Two design challenges

Sample efficiency

- Real-code runs cost time
- Large environment space
- Repetition competes with exploration

Practical usefulness

- Avoid trivial zero-capacity examples
- Allow expected adaptation delays
- Choose meaningful reference comparisons

Representing a network trace

$$T = (B, L, D, I)$$

Interval	B: bandwidth	L: one-way delay	D: duration
1	20 Mbps	20 ms	1 s
2	5 Mbps	80 ms	2 s
3	20 Mbps	10 ms	1 s

I: queue capacity and optional transfer size, fixed within a trace

Table 1 bounds: 1–100 Mbps, 5–100 ms, 0.5–2.5 s per interval

Intervals versus packet delivery opportunities

Target	Intervals	PDOs
BBR	79.11%	14.31%
CUBIC	46.24%	29.78%
Vegas	84.71%	27.29%
Reno	41.26%	28.80%

- Matched comparison inside AdvNet
- Higher PDO dimension makes search harder
- Direct control of delay changes

Higher scores expose larger throughput shortfalls. This compares representations, not complete systems.

Trace-based and action-based adversaries

Action-based adversary

- Observe state and choose the next condition online
- Requires reliable synchronization despite scheduling jitter

Trace-based adversary

- Generate the full trace before execution
- Replay the same external conditions for comparison

Genetic algorithms in AdvNet

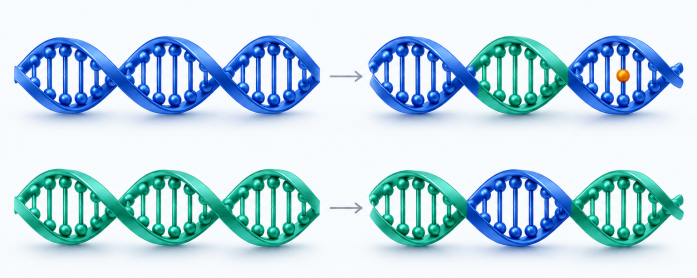
Select

Favor promising traces

Recombine

Exchange parameter segments

Candidate = a complete trace
Fitness = measured performance-gap score



Mutation introduces new values; every child still needs evaluation.

Crossover and integer mutation

Two-point crossover

	x_1	x_2	x_3	x_4
Parent A	20	5	40	10
Parent B	60	30	15	80
Child 1	20	30	15	10
Child 2	60	5	40	80

Integer mutation

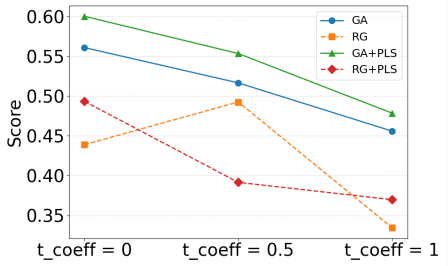
Each gene mutates with probability p.

15 → 73

Sample from that gene's allowed range.
Toy range here: 1–100.

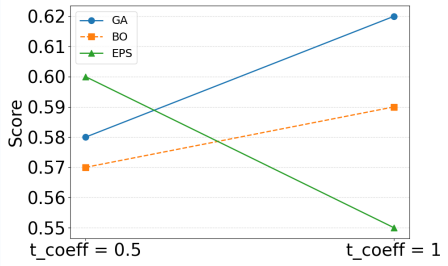
Evidence for the search design

GA versus random generation



1 hour total; PLS uses the last 6 minutes

GA, BO, and EPS



2 hours total; PLS uses the last 12 minutes

Noisy scores and SimpleMax

SimpleMax

Return the largest score observed during search.

- Hundreds of candidates create chances for lucky observations
- The apparent winner may score lower when re-tested

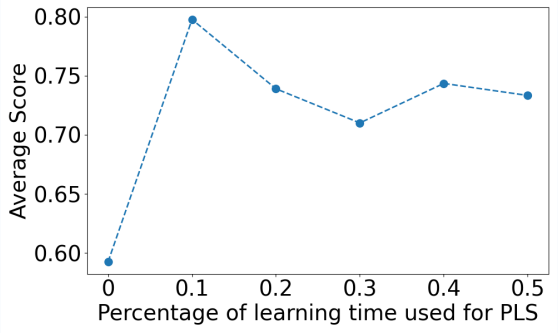
Multi-round elimination

25

Top search candidates

- Evaluate every remaining candidate once
- Remove roughly the lower half by observed mean
- At five or fewer, keep measuring until the budget ends
- Return the highest observed mean

The search and selection budget



90% search
10% PLS

- Best split in this experiment
- More selection means less exploration

From search design to practical use

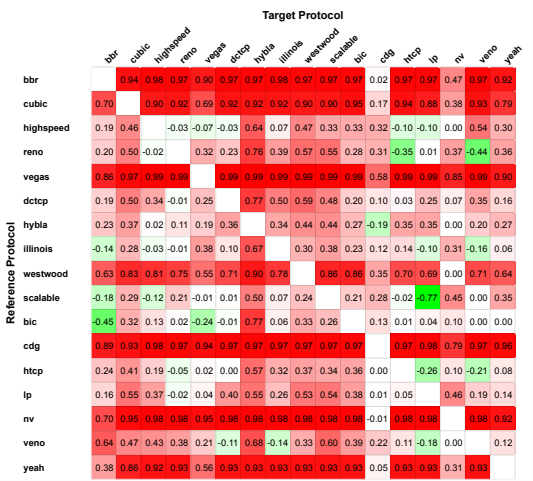
- Compact representation makes the search manageable
- GA uses execution feedback to discover challenging traces
- PLS selects an output for practical investigation

Next: Xiaoyang presents the protocol testing use cases.

Developer tasks and comparisons

Use case	Reference	Target / comparison	Question
UC1	Capacity, another TCP, or a reference flow alone	Protocol under test, or reference with a competitor	Underutilization or unfairness?
UC2	Earlier protocol version	Newer version; compare the same metric	Did an update regress?
UC3	One path	Two paths	Can an extra path hurt?

17 TCP protocols: pairwise robustness



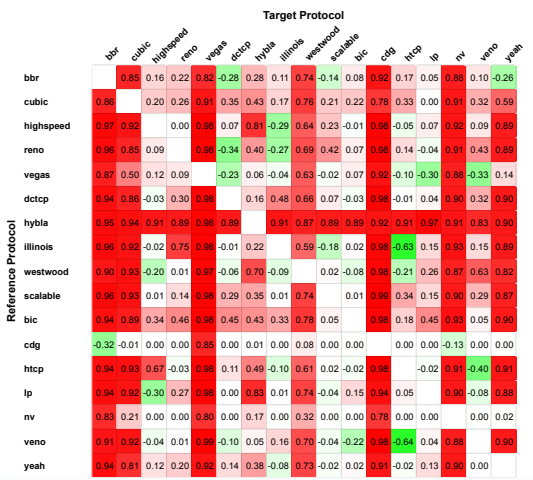
Equal throughput / delay weights

17 protocols
272 directed pairs per objective
544 scenarios across both objectives

448 / 544

scenarios with a positive score

Different traces, different weaknesses



Throughput-only objective

Reference	Target	Score
BBR	CUBIC	0.85
CUBIC	BBR	0.86

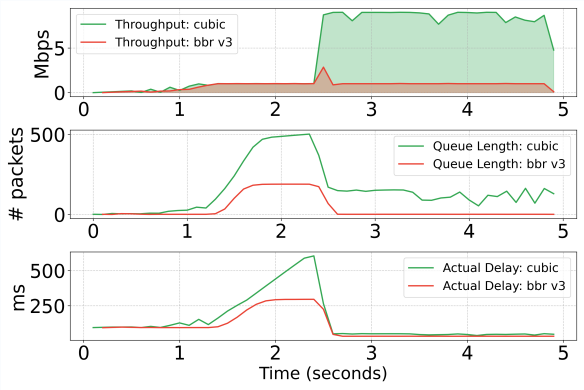
Each cell uses a different discovered trace. Both directions can reveal a weakness.

A Linux TCP acknowledgment bug

Step	Observed event
1	Delay drops; later packets arrive first
2	Three duplicate ACKs trigger fast retransmit
3	The original packet arrives before its retransmission
4	ACK advancement fails; duplicate ACKs persist
5	The sender enters retransmission timeout

eBPF and Wireshark helped the authors diagnose the packet sequence.

BBR v3: slow convergence



- Short run: BBR v3 stays low while CUBIC ramps up
- Longer run: BBR v3 eventually increases throughput
- Changing fq_codel to fq did not remove slow convergence

Fairness between competing flows

Fairness comparison

Reference score	Target score
Reference flow running alone	The same reference flow competing with a target

40–80 s

Initially normal operation, then one packet at a time per ACK for the remaining 25–60 seconds.

Regression testing: BBR v1 and v3

Dynamic trace, then 60 s fixed conditions; measure the final 10 s

Competing configuration	Reported throughput balance
CUBIC with BBR v1	CUBIC / BBR v1 = 12
CUBIC with BBR v3	BBR v3 / CUBIC = 11.6

Single-flow tests also found BBR v1 throughput more than five times BBR v3.

DChannel and early timeout estimates

Path	One-way delay	Typical packets
High bandwidth (HBP)	120 ms	Large data packets
Low latency (LLP)	3 ms	Small control packets

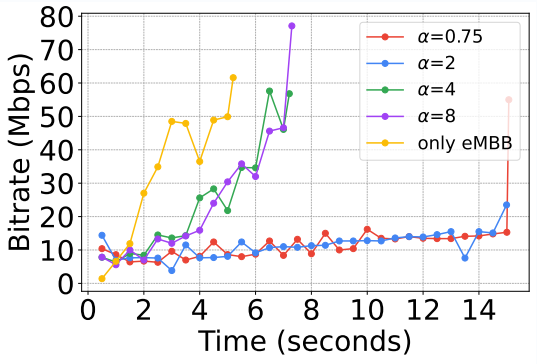
Small early RTT estimate

The handshake uses the LLP.

About 200 ms minimum RTO

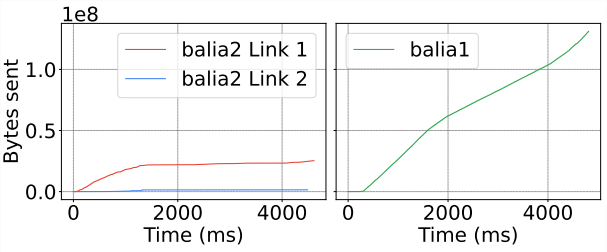
HBP data can time out before its ACK.

DChannel: two paths can finish later



- 20 MB file transfer
- All tested alpha values finish later than the single HBP
- Changing alpha softens the issue but does not remove it

MPTCP: one path versus two



- All four tested MPTCP algorithms have counterexamples
- BALIA two-path throughput is 85.5% lower in this example
- A ~40 ms ACK delay precedes the slowdown on both paths

What AdvNet contributes

- A shared search pipeline supports several developer tasks
- Compact environments and noise handling contribute to its effectiveness
- Representation, evaluation, and reference design deserve attention

Scope and open questions

Current boundaries

- No estimate of deployment frequency
- Manual root-cause diagnosis
- Limited workload and loss dimensions

Extensions

- Richer environments and workloads
- Characterize problematic environment regions
- An ABR prototype already exists

Backup: formal methods and GANs

Formal methods

- Specify properties and models
- Can provide guarantees within those assumptions
- Scaling to real kernel state is challenging

GANs

- Usually learn a data distribution
- AdvNet has no challenging-trace dataset to imitate
- Execution feedback drives a search for weaknesses

Backup: alternatives to MRE

Method	Budget allocation
RoundRobin	Equal allocation across candidates
OCBA	More measurements for uncertain or close contenders
MRE	Repeated elimination, then continued evaluation of survivors

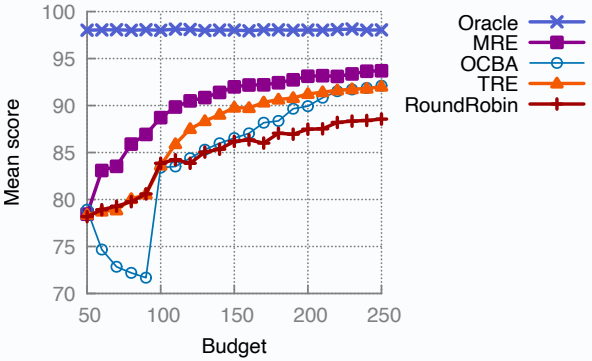
Backup: GA operator equations

$$c_i = \begin{cases} b_i, & k_1 < i \leq k_2, \\ a_i, & \text{otherwise} \end{cases}$$

$$m_i = \begin{cases} U_i, & \text{with probability } p, \\ c_i, & \text{otherwise} \end{cases} \qquad U_i \sim \mathrm{Unif}\{l_i, \ldots, u_i\}$$

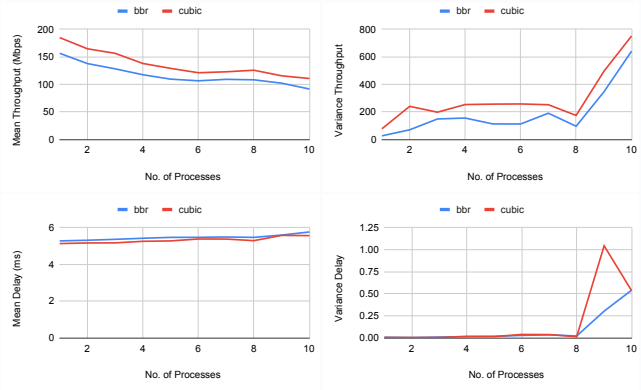
a, b: parents; c: child; m: mutated child; k_1, k_2: cut positions; l_i, u_i: bounds

Backup: the selection simulation



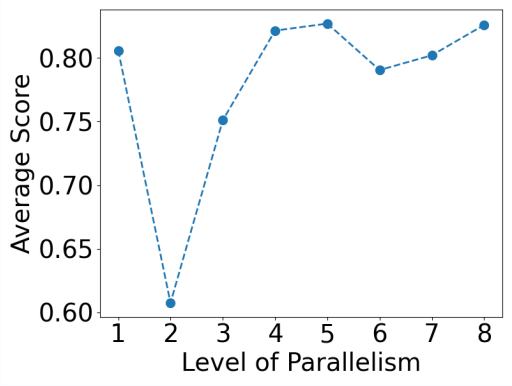
- 50 Gaussian alternatives with means sampled from 0–100
- The vertical axis is the true mean of the selected alternative
- MRE leads among the tested methods under this statistical model

Backup: parallel execution costs



- Three dedicated cores per execution
- Throughput changes with parallelism
- Variance rises at nine and ten executions

Backup: choosing the parallelism level



- More repetitions help with noise but add interference
- Five-way evaluation gives the best score in this setup
- The improvement over one evaluation is small